

1. Overview

Scenario.

You are building a backend for an application used by a Romanian insurance company that manages **building insurance policies**.

There are two main user groups:

1. **Brokers**
 - Manage clients.
 - Manage buildings (objects of insurance).
 - Create and manage policies.
 2. **Administrators**
 - Maintain metadata (currencies, fees/coefficients, risk factors).
 - Manage brokers (activate/deactivate).
 - View reports (e.g., number and total value of policies per country/county/city, per broker, per period).
-

2. Learning goals

By implementing this exercise, the learner should practice:

1. Designing clear **domain models** and relationships.
 2. Exposing **REST API endpoints** for different user roles.
 3. Implement according API first - design approach
 4. Handling **basic business rules** around policies and premiums.
 5. Writing code that is **easy to extend**: separation between domain, data access, and Web API.
 6. Implementing **unit tests** for core logic and **integration tests** for API endpoints and persistence. At least 80% code coverage is required (this percentage can grow depending on the technical stack used)
 7. Thinking about **code quality**, **readability**, and **maintainability** from the start.
-

3. Domain overview & core concepts

The system manages:

In scope for this sprint:

- **Clients** – people or companies that own buildings and buy policies.
- **Buildings** – the insured objects.
- **Geography** – country, county, city.

Out of scope for this sprint:

- **Brokers** – intermediaries that sell policies.
 - **Policies** – contracts covering buildings for a period, with a premium and currency.
 - **Administrators** – internal staff who manage configuration and reporting.
 - **Metadata** – currencies, fees/percentages, risk coefficients, etc.
-

4. Core models & relationships

Below is a **business-focused description** of models and how they relate. (Implementation details like classes and tables can be derived by developers.)

4.1 Geography

- **Country**
 - Has: Name
 - A **Country** contains many **Counties**.
- **County**
 - Has: Name
 - Belongs to one **Country**.
 - A **County** contains many **Cities**.
- **City**
 - Has: Name (you can add a county code to it to ensure it is unique)
 - Belongs to one **County**.
 - A **City** can have many **Buildings**.

Relations:

- Country (1) → (many) County
 - County (1) → (many) City
 - City (1) → (many) Building
-

4.2 Clients

- **Client**
 - Represents a **person or company**.
 - Fields:

- Unique identifier.
- Client type: Individual / Company.
- Name / full name.
- Personal Identification Number (CNP) or Company Registration Number (CUI).
- Contact info: email, phone, address.
- A **Client** can own multiple **Buildings** and multiple **Policies**.

Relations:

- Client (1) → (many) Building (a client can own multiple buildings).
 - Client (1) → (many) Policy (a client can have multiple policies over time).
-

4.3 Buildings (Object of insurance)

- **Building**
 - Describes the **object of insurance**.
 - Fields:
 - Unique identifier.
 - Owner: reference to **Client**.
 - Address (street, number).
 - Reference to **City** (from the geographic model).
 - Construction year.
 - Building type (e.g., Residential, Office, Industrial).
 - Number of floors.
 - Surface area.
 - Insured value (amount).
 - Optional risk indicators (e.g., flood zone, earthquake risk zone, etc.).
 - A **Building** can be linked to multiple **Policies** over time, but at a given moment it might be covered by 0, 1 or multiple policies (depending on rules you choose).

Relations:

- Building (many) ↔ (1) Client (owner).
 - Building (1) → (many) Policy (each policy covers exactly one).
-

5. Functional requirements – Broker-facing endpoints

These are written from a **business analyst** perspective; actual routes, HTTP verbs, etc., are at the developers' discretion.

5.1 Client management

1. Brokers must be able to **create a new client** with:
 - Client type (individual or company).
 - Name.
 - Identification number (CNP/CUI).
 - Contact details (email, phone).
 - One primary address (optional at creation).
 2. Brokers must be able to **update client details** (name, contact info) but:
 - The primary identifier (CNP/CUI) should not be easily changed, or changes should be tracked.
 3. Brokers must be able to **view client details**, including:
 - Basic info.
 - Owned buildings.
 - Existing policies.
 4. Brokers must be able to **search clients** by:
 - Name (partial match).
 - Identification number (exact match).
-

5.2 Building management

5. Brokers must be able to **register a new building** for a client:
 - Select existing client.
 - Address (street, number).
 - City (selected from existing City list).
 - Construction year.
 - Building type.
 - Surface area.
 - Insured value.
 6. Brokers must be able to **update building details**, except:
 - The linked client cannot be changed (for simplicity; or changes are restricted).
 7. Brokers must be able to **list buildings** for a given client.
 8. Brokers must be able to **view building details**, including:
 - Geography (Country, County, City).
 - Related policies.
-

6. Non-functional & code-quality requirements

The code should aim for:

1. **Separation of concerns**
 - Keep business logic out of controllers.

- Use services or application layer classes to handle business logic.
 - 2. **Maintainability**
 - Clear naming.
 - Small methods with single responsibility.
 - Avoid duplication (e.g., one coherent policy calculation service).
 - 3. **Scalability (at a basic level)**
 - Avoid loading too much data into memory unnecessarily.
 - Use paging for listing endpoints (clients, etc.), even if simple.
 - 4. **Extensibility**
 - New features should require **minimal changes**.
 - Use design patterns (e.g. Strategy) to structure your code to adapt to new features.
 - 5. **Validation**
 - Validate incoming data:
 - Required fields (e.g., client name, identification numbers, building address).
 - Reasonable ranges (e.g., positive Surface area, insured value > 0).
 - Return clear error messages.
 - 6. **Error handling and logging**
 - Centralised error handling (e.g., a middleware or equivalent).
 - Log key events: client creation, building creation, failures.
 - 7. **Security (basic level)**
 - Assume that some form of authentication will be used.
 - Endpoints for brokers and admins should be **clearly separated**, ready to apply authorization rules.
-

7. API specification (high-level)

Examples (developers can choose exact routes):

7.1 Broker – Clients

- GET /api/brokers/clients?name={name}&identifier={idNumber}
- GET /api/brokers/clients/{clientId}
- POST /api/brokers/clients
- PUT /api/brokers/clients/{clientId}

Validation examples:

- Name required.
- IdentificationNumber required and unique.
- Valid email format.

7.2 Broker – Buildings

- GET /api/brokers/clients/{clientId}/buildings
- GET /api/brokers/buildings/{buildingId}
- POST /api/brokers/clients/{clientId}/buildings
- PUT /api/brokers/buildings/{buildingId}

7.3 Broker – Geographies

- GET /api/brokers/countries
- GET /api/brokers/countries/{countryId}/counties
- GET /api/brokers/counties/{countyId}/cities

Responses should include clear status codes:

- 200 OK (success).
- 201 Created (for creations).
- 400 Bad Request (validation errors).
- 404 Not Found.
- 500 for unexpected server errors.

8. Testing plan (separate: unit & integration)

8.1 Unit tests

Goal: Verify **business logic** in isolation.

Focus on:

1. **Validation logic**
 - Client creation:
 - Missing name should fail.
 - Duplicate identification number should fail.
 - Building registration:
 - Negative insured value should fail.
 - Missing city should fail.

What to mock for unit tests:

- Data access abstractions (clients, buildings, geography).
- Logging interfaces.

8.2 Integration tests

Goal: Verify the Web API endpoints work correctly **end-to-end**: routing, model binding, validation, and database persistence.

Suggested integration tests:

1. **Create client + building flow**

- Arrange:
 - Start with empty database (or known seed: countries, counties, cities).
- Steps:
 - Create a client via API.
 - Register a building for that client.
- Assert:
 - All responses return expected HTTP status codes.
 - Client, building are persisted with correct links.

Integration test considerations:

- Use an **in-memory or test-specific database**.
 - Run setup before tests.
 - Clean database state between tests.
-

10. Acceptance criteria (checklist)

For the exercise to be considered complete:

1. **Core CRUD**
 - Brokers can create and manage clients, buildings via API..
 2. **Geography**
 - Buildings are linked to City, County, and Country.
 - Reports correctly group policies by these geographic levels.
 3. **Quality**
 - Code is structured in clear layers (domain, application/services, infrastructure, Web).
 - Validation is implemented and tested.
 - Centralised error handling present.
 4. **Tests**
 - Unit tests cover clients, buildings management.
 - Integration tests cover main flows limited to the current scope.
 - Tests are easy to run from the solution.
-

